

Hubitat_v2 – Crestron-Hubitat Integration Driver Tutorial

The original Crestron-Hubitat driver included separate integration modules for switches, lights, thermostats, robotic vacuums, etc. The new, Hubitat_v2, version of the driver takes a different approach and includes just seven modules:

1. **Hubitat_v2 Comm Manager:** The Comm Manager module establishes communications between a Crestron processor and a Hubitat Elevation Hub. It also configures the Hubitat's MakerAPI app, that is used for integration, to send device update information to the Crestron processor.
2. **Hubitat_v2 Device:** The device module registers a Hubitat connected device that is to be integrated with a Crestron system and obtains its current status.

Instead of a module for each type of device, the new driver uses modules for the different attributes that comprise devices. For example, to integrate a typical thermostat with a Crestron system requires control of the operating mode of the system (heat, cool, etc.), the fan mode of a forced air system (auto, on), the heating/cooling set point, and knowing what the temperature sensor in the thermostat is reporting. The Hubitat_v2 modules needed to integrate the attributes of devices are:

3. **Hubitat_v2 Enum:** In computer programming, an enum (short for enumeration) is a distinct set of named choices. For example, the choices for the operating modes of a typical HVAC system are heat, cool, auto, and off. The Hubitat_v2 Enum module allows the Crestron program to choose between these choices to control the HVAC system's operating mode. A second Enum module is used to control the fan mode. Additional Enum modules can be included in a Crestron program to control the features of other smart home devices.
4. **Hubitat_v2 Numeric:** The numeric module is used to set or read any numeric value. One is used to adjust the set point of a thermostat while a second one is used to obtain the value from the thermostat's temperature sensor. If a Crestron programmer is working with a thermostat with a humidity sensor, they just need to add another numeric module to their Crestron program.
5. **Hubitat_v2 String:** The string module is used to set or read any string value. For example, the current operating state of a robotic vacuum or the error state of the vacuum if it gets stuck, its brush is tangled, the vacuum is full, etc.

Finally, there are two additional modules to complete the new Hubitat_v2 integration driver.

6. Hubitat_v2 Command: The command module is used to trigger an action. For example, it can be used to trigger a lighting scene defined on a Hubitat Elevation Hub.
7. Hubitat_v2 Modes: The mode module is a special version of the Enum module that is used to select between the different operating modes defined on a Hubitat Elevation Hub. These are typically: Day, Evening, Night, and Away, but can be changed by a user. Modes are global and can be used to trigger different behaviors on a hub in response to button presses, sensor inputs, etc.

This creates a much more flexible system for Crestron programmers than was available to them with the original Crestron-Hubitat driver. Instead of the programmer having to request that support for a new device type be added to the integration driver, or requesting that a new product feature be supported by an existing module, the new driver is designed to provide the programmer with the tools to easily accomplish this themselves.

A “demo program” is included with the driver. It provides examples of support for several common off-the-shelf smart home devices that could be paired with a Hubitat Elevation Hub and integrated with a Crestron system. From the demo program, below is a simple example of how support for a basic smart switch is developed, based on the JSON provided by the Hubitat MakerAPI app (MakerAPI is an optional application that a user can choose to run on their hub that provides a mechanism for integrating the hub with a 3rd party platform). The steps are as follows:

You will first need to install the Maker API app on your Hubitat Elevation Hub. Hubitat provides instructions for doing this and they can be found [here](#). You only need to follow the first 7 steps shown in the instructions. The remainder of the instructions go into the details on how a third-party system can communicate with a Hubitat through the Maker API. For those interested, this is how the Crestron-Hubitat integration driver works.

The MakerAPI app web interface provides a number of URLs for obtaining information about the devices paired with the Hubitat that the user wants to expose to third party platforms through the MakerAPI app. The first URL in the list provides a JSON formatted catalog of all the devices being exposed through the MakerAPI to third party platforms along with their device id's. A device id is included as a parameter in Hubitat_v2 modules to link the modules to a specific device. Here is an example of the returned JSON:

```
[
{
  "id": "100",
  "name": "Coffee Maker Switch",
  "label": "Coffee Maker Switch",
  "type": "Generic Zigbee Outlet",
  "room": "Kitchen",
  "roomId": 1,
```

```

    "notes": ""
  },
  {
    [JSON for another device]
  }
]

```

Including the device id of 100 in the third URL shown in the MakerAPI web interface, which returns the detailed information for a specific device, returns the following JSON:

```

{
  "id": "100",
  "name": "Coffee Maker Switch",
  "label": "Coffee Maker Switch",
  "type": "Generic Zigbee Outlet",
  "room": "Kitchen",
  "roomId": 1,
  "notes": "",
  "attributes": [
    {
      "name": "switch",
      "currentValue": "off",
      "dataType": "ENUM",
      "values": [
        "on",
        "off"
      ]
    },
    {
      "name": "power",
      "currentValue": null,
      "dataType": "NUMBER"
    },
    {
      "name": "switch",
      "currentValue": "off",
      "dataType": "ENUM",
      "values": [
        "on",
        "off"
      ]
    }
  ],
  "capabilities": [

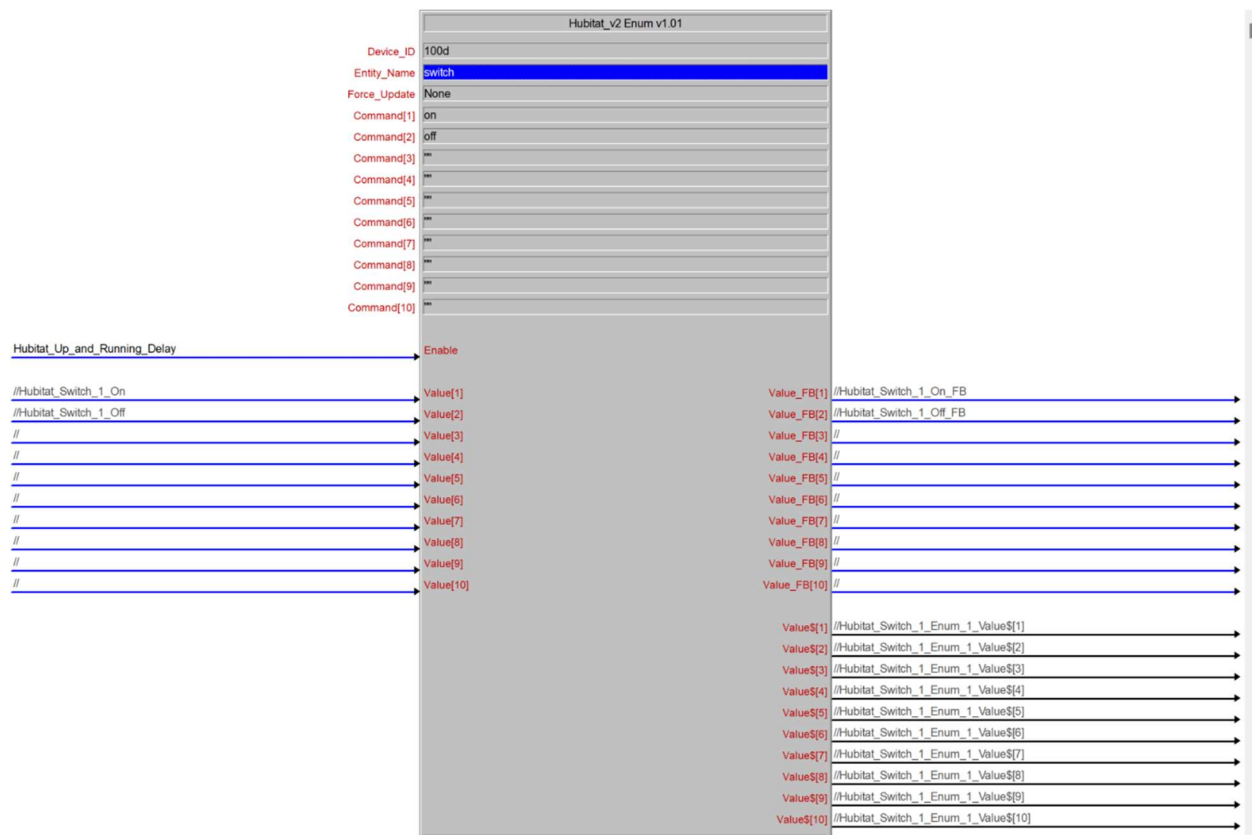
```

```

"Configuration",
"Actuator",
"Refresh",
"PowerMeter",
{
  "attributes": [
    {
      "name": "power",
      "dataType": null
    }
  ]
},
"Outlet",
{
  "attributes": [
    {
      "name": "switch",
      "dataType": null
    }
  ]
},
"Switch",
{
  "attributes": [
    {
      "name": "switch",
      "dataType": null
    }
  ]
},
"Sensor"
],
"commands": [
  "configure",
  "flash",
  "off",
  "on",
  "refresh",
  "updateFirmware"
]
}

```

There are just three things that are important for a Crestron programmer to look at in the above JSON:



First, there is an ENUM section in the JSON that shows that a Hubitat_v2 Enum module will be needed to turn the smart plug on/off. Second, the values (on, off) in the ENUN section show that the Value[1] input on the Crestron module will be used to turn the switch on and Value[2] will turn it off. Similarly, the Value_FB[1] output of the Crestron module will be high if the switch is on and Value_FB[2] will be high if the switch is off. The order of the values in the JSON will match the order of the functionality of the Value[] inputs and outputs of the Hubitat_v2 Enum module.

Second, the Crestron programmer must include the commands from the bottom of the JSON as parameters in the Hubitat_v2 Enum module. The command “on” must be entered in the Command[1] parameter field so it aligns with the Value[1] module input. Then, the on command will be sent to the Hubitat when the Value[1] input is pulsed. The same holds true for the off command.

Finally, the Crestron programmer must include the name of the ENUM, in this case “switch” in the Entity_Name parameter field of the Enum module. This tells the module to look for update messages for a switch with the selected device id. A message that matches both these fields will be used to drive the state of the module’s Value_FB[] outputs.

While this part may be a bit confusing at first, by studying how different devices are implemented in the demo program it should quickly become much easier to understand. In the

demo program, the JSON describing a device's attributes is included as extended help in the folder for each device so it can be compared to how the modules are configured for control of that device.

One word of caution; study the JSON provided by the MakerAPI get request carefully. I have a sensor in my master bathroom that monitors for a rise in humidity (when someone takes a shower) and turns on the bathroom fan. That sensor returned a numeric value and it was easy to drive the logic to control the bathroom fan. The sensor died and I replaced it with a very similar sensor from another manufacturer. Nothing was working. I finally figured out that while the first sensor returned a numeric value. The new sensor returned the numeric value in the form of a string. `{value:"50"}` instead of `{value:50}`. So, I had to switch from a Hubitat_v2 Numeric module to a Hubitat_v2 String module in my SimplWindows program and then add some code to convert the string to a true numeric value. Once I did that, everything started working perfectly again.